Remix, Don't Expand: Chunked Template Routing for Compute-Efficient Transformers

Anonymous Author(s)
Affiliation
Address
email

Abstract

Scaling Transformer language models by widening dense linear layers increases both parameter count and inference cost proportionally. Mixture-of-Experts (MoE) architectures decouple these by routing tokens to discrete expert subnetworks, but introduce load-balancing complexity and often underperform dense baselines at small-to-medium scale. We introduce **RemixedLinear**, a drop-in replacement for standard linear layers that dynamically mixes K learned template matrices per chunk of tokens using lightweight content-aware routing. Each token's effective linear operator is a context-dependent combination of shared templates, gated by a centered output gate $(1 + \tanh)$ that preserves dense-equivalent outputs from initialization. Routing is amortized over 64-token chunks and balanced via quantile tracking, eliminating the auxiliary losses required by standard MoE. Through 29 phases of systematic ablation, we identify the key design choices that enable dynamic linear layers to outperform static baselines: chunk-amortized routing, quantile balancing, and identity-preserving initialization. At depth 12 ($\sim$290M active parameters), RemixedLinear achieves **0.814 bits-per-byte** compared to 0.903 for the dense baseline at matched active FLOPs, a 10% improvement. Standard MoE with equivalent routing underperforms the dense baseline. These results hold across three model scales (d4, d8, d12) following a consistent power-law improvement over dense scaling curves.

1 Introduction

Linear transformations dominate both the parameter count and computational cost of Transformer language models [Vaswani et al., 2017]. As models scale, improving the efficiency of these layers becomes critical for practical deployment. Two broad strategies have emerged: static compression methods such as low-rank factorization [Hu et al., 2022], which reduce parameters but restrict all inputs to a fixed subspace; and sparse routing methods such as Mixture-of-Experts [Shazeer et al., 2017, Fedus et al., 2022], which activate only a subset of parameters per token but introduce routing instability, load-balancing overhead, and often require substantially larger total parameter counts.

We introduce **RemixedLinear**, a linear layer that stores K learned template matrices and dynamically composes them per chunk of tokens using content-aware routing. Unlike standard MoE, which routes entire tokens to discrete expert subnetworks at the feed-forward network (FFN) level, RemixedLinear operates at the level of individual linear projections (Q, K, V, projection, and both FFN layers), providing finer-grained dynamic computation. Unlike static low-rank layers, each token induces a distinct full-rank effective operator via continuous template mixing.

The design of RemixedLinear was shaped by 29 phases of systematic ablation. Early designs based on content-dependent multiplicative gating consistently failed due to what we term *optimization friction*: the gradient cost of simultaneously learning routing and feature weights exceeded the

Submitted to 40th Conference on Neural Information Processing Systems (NeurIPS 2026). Do not distribute.

capacity benefit of dynamic computation. The key innovations that resolved this are: (1) **chunk routing**, which amortizes a single routing decision (anchored on the first token of each chunk to preserve causality) over $N{=}64$ tokens, reducing per-token routing overhead and stabilizing gradients; (2) **quantile-balanced routing**, which replaces auxiliary load-balancing losses with EMA-tracked quantile normalization; and (3) a **centered output gate** $(1 + \tanh)$, initialized to pass-through, ensuring the model starts with dense-equivalent outputs.

We evaluate RemixedLinear on the nanochat framework [Karpathy, 2025] using the ClimbMix dataset with Chinchilla-optimal training horizons [Hoffmann et al., 2022]. At depth 12 ($\sim$290M active parameters), RemixedLinear achieves 0.814 bits-per-byte (BPP) compared to 0.903 for the dense baseline at matched active FLOPs. Standard MoE at depth 4 scores 1.187 BPP, worse than the dense baseline of 1.17. These results hold across three model scales, with RemixedLinear points consistently falling below the dense power-law scaling curve when plotted against active compute.

2 Related Work

Mixture-of-Experts. MoE architectures [Shazeer et al., 2017, Lepikhin et al., 2021, Fedus et al., 2022] route tokens to discrete expert subnetworks, enabling large parameter counts with constant inference FLOPs. However, MoE introduces routing instability, expert collapse, and hardware inefficiency from sparse memory access. Expert Choice routing [Zhou et al., 2022] and ST-MoE [Zoph et al., 2022] address load balancing but retain the coarse granularity of routing entire tokens to complete FFN blocks. RemixedLinear shares the goal of decoupling stored and active parameters but operates at finer granularity (individual linear projections) and uses continuous mixing rather than discrete routing.

Dynamic Weights and Hypernetworks. Hypernetworks [Ha et al., 2017] generate weights conditioned on context but require $O(d^2)$ operations per token to produce a full weight matrix. Fast Weight Programmers [Schlag et al., 2021] explore similar principles for attention. RemixedLinear generates only $O(K)$ routing coefficients to mix pre-existing template matrices, achieving dynamic weights with negligible overhead.

Parameter-Efficient Adaptation. LoRA [Hu et al., 2022] uses static low-rank decompositions for efficient fine-tuning. RemixedLinear extends this principle to the dynamic regime: the basis projection provides compression while template routing provides token-dependent expressivity that static decompositions cannot achieve.

Gated Linear Units. GLU variants [Dauphin et al., 2017, Shazeer, 2020] gate neuron activations after a fixed linear transformation. RemixedLinear gates at a different level: it modulates which *template combination* constructs the effective linear operator, then applies a separate low-rank output gate. This produces a family of context-dependent transformations rather than a single gated projection.

Scaling Laws. Chinchilla scaling laws [Hoffmann et al., 2022] establish compute-optimal training horizons as a function of model size. We adopt this framework, training each model with a fixed token-to-parameter ratio, enabling fair comparison across scales. Our results demonstrate that RemixedLinear shifts the scaling curve, achieving lower BPP at matched active FLOPs.

3 Method

3.1 Overview

A standard linear layer computes $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$, where $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ is a static matrix applied identically to all inputs. RemixedLinear replaces this with a dynamic composition:

$$\mathbf{y} = (\mathbf{W}_{\text{eff}}(\mathbf{c}) \cdot \text{LN}(\mathbf{W}_b\mathbf{x})) \odot \mathbf{g}_{\text{out}}(\mathbf{c}) + \mathbf{b} \tag{1}$$

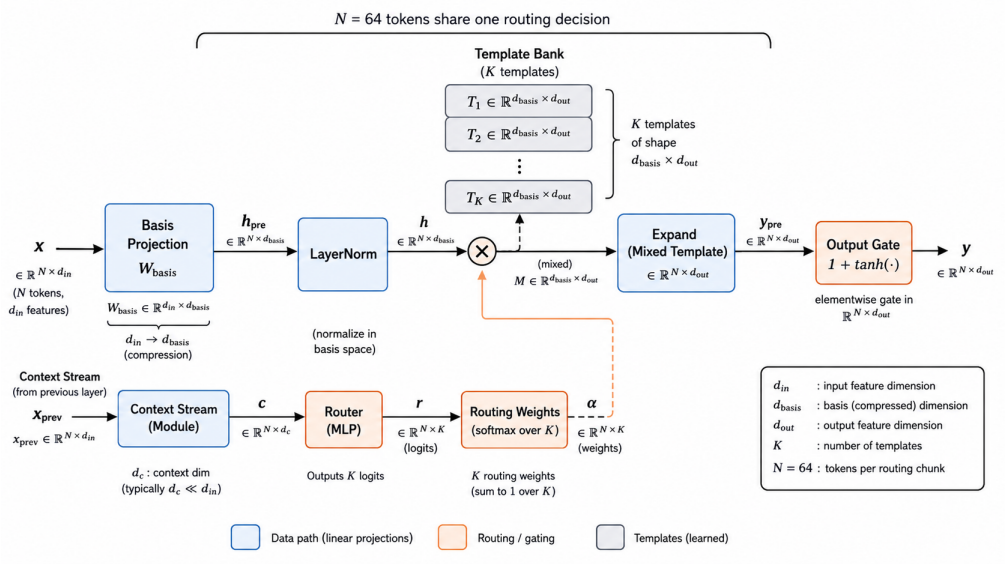

Figure 1: Architecture of the RemixedLinear layer. The data path (blue) compresses input $\mathbf{x}$ via a basis projection, normalizes it, then expands through a mixed template matrix. The context stream (orange) produces routing weights $\boldsymbol{\alpha}$ that select the template combination from a bank of K learned templates (gray). Routing is amortized over chunks of N=64 tokens. A centered output gate $(1 + \tanh)$ modulates the final output.

where $\mathbf{W}_b \in \mathbb{R}^{B \times d_{\text{in}}}$ is a basis projection (B = basis size), $\mathbf{W}_{\text{eff}}(\mathbf{c})$ is a context-dependent effective mixing matrix constructed from K templates, and $\mathbf{g}_{\text{out}}(\mathbf{c})$ is a low-rank output gate. The context signal $\mathbf{c}$ is derived from a causal context stream (Section 3.4).

3.2 Template Mixing with Chunk Routing

RemixedLinear stores K template matrices $\{\mathbf{T}_k\}_{k=1}^{K}$, each $\mathbf{T}_k \in \mathbb{R}^{d_{\text{out}} \times B}$. The effective mixing matrix is a weighted combination:

$$\mathbf{W}_{\text{eff}}(\mathbf{c}) = \sum_{k=1}^{K} \alpha_k(\mathbf{c}) \cdot \mathbf{T}_k \tag{2}$$

where $\boldsymbol{\alpha}(\mathbf{c}) = \text{softmax}(\mathbf{W}_r \mathbf{c}/\tau)$ are routing weights computed from the context signal, $\mathbf{W}_r \in \mathbb{R}^{K \times d_{\text{in}}}$ is a learned routing projection, and τ is a temperature parameter.

Chunk amortization. Per-token routing introduces two problems: high FLOPs overhead from computing $\boldsymbol{\alpha}$ for every token, and gradient instability from the multiplicative coupling between routing weights and template gradients. We amortize routing over chunks of N=64 tokens: the routing weights $\boldsymbol{\alpha}$ are computed once from the *first token* of each chunk and shared across all tokens in that chunk. Because the anchor is the chunk's earliest token, no future information leaks into the routing decision, preserving the autoregressive causal constraint. This reduces routing FLOPs by a factor of N and stabilizes training by decoupling routing updates from per-token gradient noise.

Quantile-balanced routing. Standard MoE requires auxiliary load-balancing losses to prevent expert collapse [Fedus et al., 2022]. We replace this with quantile-balanced routing. Concretely, we maintain a per-expert EMA threshold $\boldsymbol{\theta}_k$ updated each step as $\boldsymbol{\theta}_k \leftarrow \lambda\, \boldsymbol{\theta}_k + (1{-}\lambda)\, q_k$, where q_k is the $(1 - \text{topk}/K)$-quantile of expert k's batch scores and λ=0.99. A token is assigned to expert k if its score exceeds $\boldsymbol{\theta}_k$, unioned with a hard top-k fallback to guarantee at least k active experts per token; excess selections are trimmed to the highest-scoring k. The resulting mask is applied before a masked softmax, yielding differentiable routing weights. This ensures balanced template utilization without introducing an additional loss term or its associated hyperparameter.

Table 1: FLOPs per token for a single linear projection with $d_{\text{in}} = d_{\text{out}} = d$, basis size $B = d$, K templates, chunk size N, context dimension d_c, gate rank r.

Component	**FLOPs**
Dense baseline ($\mathbf{W}\mathbf{x}$)	$2d^2$
Basis projection ($\mathbf{W}_b\mathbf{x}$)	$2dB$
Template assembly ($\sum \alpha_k \mathbf{T}_k$, amort.)	$2KBd/N$
Template mixing ($\mathbf{W}_{\text{eff}}\mathbf{h}$)	$2Bd$
Routing (amortized, $1/N$)	$2d_cK/N$
Output gate (rank r)	$2(d_cr + rd)$
Total (RemixedLinear)	$\approx 4dB + 2r(d_c{+}d)$

3.3 Centered Output Gate

A low-rank output gate modulates the output:

$$\mathbf{g}_{\text{out}}(\mathbf{c}) = 1 + \tanh\big(s \cdot (\mathbf{W}_c\mathbf{c})^\top \mathbf{G}\big) \tag{3}$$

where $\mathbf{W}_c \in \mathbb{R}^{r\times d_c}$ projects the context vector to r coefficients, $\mathbf{G} \in \mathbb{R}^{r\times d_{\text{out}}}$ is a learned basis, d_c is the context stream dimension, $r{=}8$ is the gate rank, and s is a learned scalar initialized to 0.1. The $1 + \tanh(\cdot)$ formulation, with $\mathbf{W}_c$ and $\mathbf{G}$ zero-initialized, ensures $\mathbf{g}_{\text{out}} = \mathbf{1}$ at initialization. This is critical: it guarantees that RemixedLinear produces *output-equivalent* results to a standard linear layer at step 0 (up to the intermediate LayerNorm; see Section 5), avoiding the optimization friction that caused all sigmoid-gated designs to fail in our ablation studies (Appendix A).

3.4 Context Stream

The context signal $\mathbf{c}$ is derived from a causal context stream that maintains a running representation across the sequence. We use a *selective* stream with GRU-style gating:

$$\mathbf{c}_t = (1 - \mathbf{z}_t) \odot \mathbf{c}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{c}}_t \tag{4}$$

where $\mathbf{z}_t = \sigma(\mathbf{W}_z\mathbf{x}_t)$ is an input-dependent update gate ($\mathbf{W}_z \in \mathbb{R}^{d_c\times d}$) and $\tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_w\mathbf{x}_t)$ is the candidate state ($\mathbf{W}_w \in \mathbb{R}^{d_c\times d}$, zero-initialized so that $\tilde{\mathbf{c}}_0 = \mathbf{0}$ at initialization). The context state is *detached* between steps to prevent gradient explosion through long recurrent chains, a lesson from Phase 3 of our ablation study (Appendix A).

3.5 Integration into the Transformer Block

RemixedLinear replaces all six linear projections in each Transformer block: Q, K, V, attention output projection, and both FFN layers ($\mathbf{W}_{\text{fc}}$, $\mathbf{W}_{\text{proj}}$). Each projection has its own set of K templates and routing weights, allowing different projections to specialize independently. The context stream is shared within each block.

3.6 Complexity Analysis

Table 1 compares per-token FLOPs for a single projection.

With $B = d$ (full-rank basis, as used in all experiments), RemixedLinear uses approximately $4d^2$ active FLOPs per projection versus $2d^2$ for dense, a $2\times$ per-projection overhead. However, the model stores K template matrices totaling $K \cdot B \cdot d$ parameters while the active computation per token uses only the weighted sum of templates plus the basis projection. The ratio of stored parameters to per-token active parameters is $(K \cdot B \cdot d + B \cdot d)/(B \cdot d + B \cdot d) = (K{+}1)/2$ for full-rank $B{=}d$, enabling the model to express a richer family of transformations from a larger parameter bank while maintaining bounded inference cost.

Table 2: Model configurations. RemixedLinear uses K=8 templates, chunk size N=64, quantile routing (λ=0.99), selective context stream, and full-rank basis B=d.

	d4	**d8**	**d12**
Layers (n_{layer})	4	8	12
Model dim (d)	256	512	768
Basis size (B)	256	512	768
Heads	2	4	6
Dense params	∼37M	∼126M	∼286M
Remix total params	56M	276M	792M
Remix active params	37M	127M	290M
Training tokens	∼0.4B	∼1.3B	∼3.0B

4 Experiments

4.1 Setup

Framework. All experiments use nanochat [Karpathy, 2025], a single-file GPT implementation supporting Flash Attention [Dao et al., 2022], bf16 mixed precision, and the Muon optimizer [Jordan et al., 2025] for matrix-shaped parameters with AdamW for embeddings and scalars.

Dataset. ClimbMix [NVIDIA, 2024], a curated pretraining mixture, tokenized with a 32,768-vocabulary BPE tokenizer. Sequence length is 2048 tokens.

Training horizon. Following Chinchilla scaling [Hoffmann et al., 2022], each model is trained with a fixed 10.5× token-to-scaling-parameter ratio, where scaling parameters exclude embedding tables. This ensures compute-optimal training at every scale.

Metric. Validation bits-per-byte (BPP), a vocabulary-size-invariant measure of language model quality. Lower is better.

Model scales. We evaluate at three scales. In nanochat, the "depth" parameter serves dual duty: it sets both the number of Transformer layers ($n_{\text{layer}} = \text{depth}$) and the base model width ($d = \text{depth} \times 64$, rounded up to the nearest multiple of the head dimension 128). All experiments use full-rank basis $B = d$, context dimension $d_c = d$, output gate rank $r = 16$, and head dimension 128.

Baselines. (1) Dense Transformer baseline (nanochat default) at depths 12 through 30. (2) Standard MoE with 8 experts, top-1 and top-all routing at depth 4.

4.2 Main Results

Table 3 presents the primary comparison.

At every scale, RemixedLinear achieves lower BPP than the dense baseline at comparable active FLOPs. The improvement grows with scale: 7.5% at d4, 6.8% at d8, and 9.9% at d12. Notably, RemixedLinear at d8 (0.903 BPP, 127M active params) matches the dense d12 baseline (0.903 BPP, 286M params), achieving equivalent quality with 2.25× fewer active parameters.

Standard MoE at d4 performs *worse* than the dense baseline (1.187 vs. 1.170 BPP), despite having 51M total parameters. This suggests that coarse-grained expert routing at the FFN level is insufficient at this scale, while RemixedLinear's fine-grained template routing within individual projections provides a qualitatively different advantage.

4.3 Scaling Law Analysis

Figure 2 plots BPP against total parameters, active parameters, and active FLOPs. The dense baseline (d12–d30) follows a tight power law $\text{BPP} = a \cdot \text{FLOPs}^{b}$. RemixedLinear points fall consistently below this curve when measured against active FLOPs, confirming that the architecture achieves genuine compute efficiency gains rather than merely trading parameters for quality.

Table 3: Main results. RemixedLinear consistently outperforms the dense baseline at matched active FLOPs across all three scales. Standard MoE underperforms the dense baseline at d4.

Model	Total Params	Active Params	FLOPs	BPP
Dense baselines				
Dense d4	37M	37M	7.8e7	1.170
Dense d8	126M	126M	2.9e8	0.969
Dense d12	286M	286M	7.6e8	0.903
RemixedLinear (ours)				
Remix d4	56M	37M	8.0e7	**1.082**
Remix d8	276M	127M	2.9e8	**0.903**
Remix d12	792M	290M	7.6e8	**0.814**
Standard MoE (d4 only)				
MoE top-1	51M	37M	7.8e7	1.187
MoE top-all	51M	51M	1.7e8	1.186

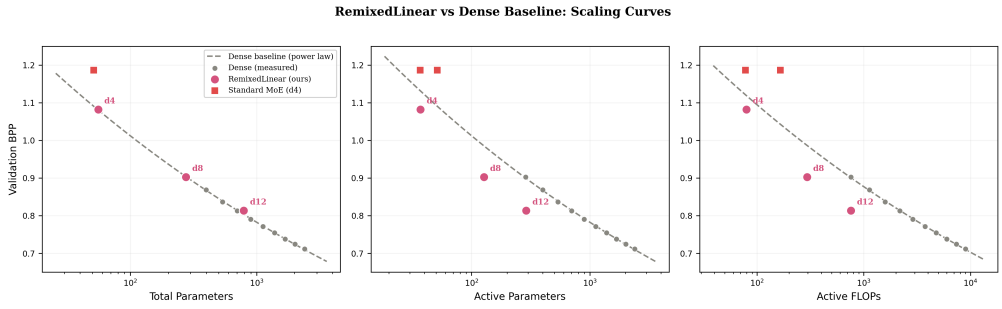

Figure 2: Validation BPP vs. total parameters (left), active parameters (center), and active FLOPs (right). Dashed lines show power-law fits to dense baselines (d12–d30). RemixedLinear points (pink circles) fall consistently below the dense scaling curve, indicating better compute efficiency. Standard MoE (red squares) falls above the curve at d4.

The d12 RemixedLinear point is particularly striking: at nearly identical active FLOPs to the dense d12 baseline (7.58×10^8 vs. 7.60×10^8), it achieves 0.814 BPP versus 0.903, matching the dense power-law extrapolation at approximately d20 scale ($\sim$900M params, $\sim 2.9 \times 10^9$ FLOPs).

4.4 Ablation Study

To isolate the contribution of each design component, we run ablations at d4 using the P29 sweep configuration. Table 4 reports results for the key variants tested.

Chunk routing matches per-token. Configurations 29A (per-token top-1) and 29C (chunk-64) achieve nearly identical BPP (1.087 vs. 1.082), while chunk routing uses substantially fewer routing FLOPs. The combined variant (29E) shows no additional benefit, confirming that chunk-64 captures the useful routing structure.

Template count matters. Reducing from $K{=}8$ to $K{=}4$ templates (29H) degrades BPP from 1.082 to 1.127, though it still outperforms the dense baseline. This suggests the model benefits from a diverse set of template specializations.

Gate contributions are modest but additive. With a single template ($K{=}1$), the basis-mixing factorization alone already matches the dense baseline (1.168 vs. 1.170). Note that the $K{=}1$ variant includes an intermediate LayerNorm between the basis projection and the mixing matrix (Eq. 1), which is absent from the dense baseline; we investigate this confound in Section 5. Adding the output gate and context stream improves BPP to 1.165, and further adding a linear basis gate reaches 1.160. The basis gate thus contributes only 0.005 BPP improvement at significant FLOPs cost ($\sim 0.625d^2$

Table 4: Ablation study at d4. *Top*: routing variants with $K{=}8$ templates. *Middle*: gate component ablation with $K{=}1$ template (isolating gate effects from routing). *Bottom*: standard MoE baselines. Δ is relative to the dense baseline BPP of 1.170.

Configuration	**BPP**	Δ
Dense baseline	1.170	–
Routing variants ($K{=}8$ templates)		
29A: Top-1 routing (per-token)	1.087	-7.1%
29C: Chunk-64 routing	**1.082**	-7.5%
29E: Top-1 + Chunk-64	1.082	-7.5%
29H: 4 templates, dense mix	1.127	-3.7%
Gate component ablation ($K{=}1$)		
No context, no gates	1.168	-0.2%
Output gate + context only	1.165	-0.4%
+ Linear basis gate	1.160	-0.9%
Standard MoE ($K{=}8$ experts)		
MoE top-1	1.187	$+1.5\%$
MoE top-all	1.186	$+1.4\%$

Table 5: Inference throughput at d12 on a single H200 GPU (batch 64, seq 2048). Hardware FLOPs are measured via `torch.utils.flop_counter`. RemixedLinear achieves 10% better BPP but at $3.7\times$ lower throughput. GPU utilization (measured TFLOPS) drops from 195 to 86, suggesting the GPU's compute units are substantially underutilized in the RemixedLinear forward pass.

Metric	**Dense d12**	**Remix d12**
Total params	286M	792M
Active params	286M	290M
HW FLOPs/token	2.2×10^8	3.6×10^8
Throughput (tok/s)	886k	242k
Peak memory (GB)	49.3	53.5
GPU utilization (TFLOPS)	195	86
Val BPP	0.903	0.814

per projection); we omit it in the final architecture, relying on the output gate and context stream for modulation while template routing provides the primary performance gain.

Standard MoE fails at this scale. Both MoE variants underperform the dense baseline, indicating that coarse expert routing is insufficient when the individual expert capacity is limited. We note that the MoE baselines were trained with a token budget computed from *total* parameters (matching the dense baseline's Chinchilla ratio); preliminary results with active-parameter-matched budgets showed similar trends but are not included here.

4.5 Inference Throughput

While RemixedLinear improves quality at matched *active* FLOPs, its wall-clock inference throughput is lower than the dense baseline. Table 5 reports measurements on a single NVIDIA H200 GPU at batch size 64, sequence length 2048, without `torch.compile`.

The hardware FLOP ratio is only $1.6\times$ (3.6e8 vs. 2.2e8), yet throughput differs by $3.7\times$. The GPU utilization drops from 195 to 86 TFLOPS, indicating that RemixedLinear's compute units are substantially idle during inference. Several factors likely contribute: the factored template-mixing computation replaces one large GEMM with multiple smaller operations (template stacking, chunk-level einsum for W_{eff}, and a second einsum for the output), which may reduce arithmetic intensity; the $K\times$ larger total parameter count increases HBM traffic; and the chunk-wise control flow may limit parallelism. We have not yet performed detailed kernel-level profiling to isolate the relative contribution of each factor.

Table 6: CORE benchmark scores (22-task centered accuracy aggregate). Higher is better. RemixedLinear achieves CORE scores 34–36% above the dense power-law prediction at matched active FLOPs, confirming that BPP improvements transfer to downstream task performance.

Model	Active Params	Active FLOPs	Val BPP	CORE
Dense d12	286M	7.6e8	0.903	0.114
Dense d14	399M	1.1e9	0.869	0.148
Dense d20	897M	2.9e9	0.791	0.215
Dense d26	1.68B	6.0e9	0.738	0.266
Dense d30	2.40B	9.0e9	0.712	0.291
Remix d8	127M	2.9e8	0.903	**0.123**
Remix d12	290M	7.6e8	0.814	**0.172**

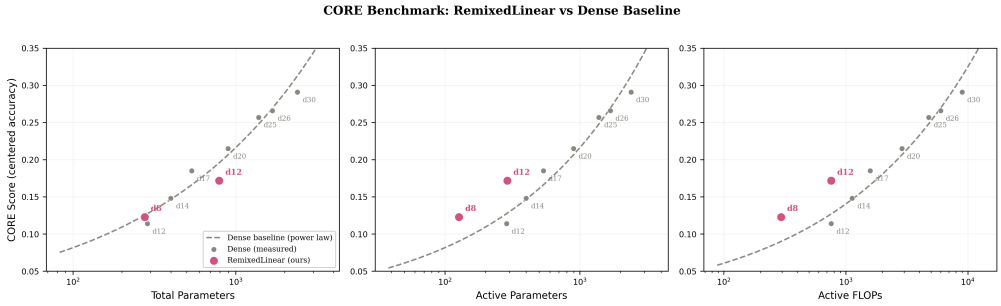

Figure 3: CORE benchmark score vs. model size and compute. **Left:** Total parameters — RemixedLinear points fall on the dense curve, reflecting the $K\times$ storage cost. **Middle:** Active parameters — RemixedLinear achieves 34–36% higher CORE scores at matched active params. **Right:** Active FLOPs — same advantage, confirming that the scaling improvement is genuine and not an artifact of the parameter-counting methodology.

Closing this throughput gap is the primary practical challenge for deployment. Promising directions include fusing the template assembly and output computation into a single GPU kernel (eliminating intermediate materializations), quantizing template weights to reduce memory traffic, and distilling to fewer templates ($K{=}1$) at inference time.

4.6 Downstream Task Evaluation

To verify that BPP improvements translate to genuine downstream task performance, we evaluate all models on the CORE benchmark [Li et al., 2024], a standardized suite of 22 in-context learning tasks from the DataComp for Language Models (DCLM) framework. The CORE metric uses *centered accuracy*: each task's raw accuracy is linearly rescaled so that random guessing maps to 0 and perfect accuracy to 1, enabling meaningful averaging across tasks with different baselines.

The 22 tasks span five categories:

- **Commonsense reasoning:** HellaSwag (0-shot and 10-shot), PIQA, WinoGrande, Winograd, COPA, CommonsenseQA, OpenBookQA
- **World knowledge:** ARC-Easy, ARC-Challenge, Jeopardy, BigBench QA WikiData
- **Reading comprehension:** BoolQ, SQuAD, CoQA, LAMBADA
- **Algorithmic reasoning:** BigBench Dyck Languages, BigBench CS Algorithms, BigBench Operators, BigBench Repeat Copy Logic
- **Other:** AGI Eval LSAT-AR (logical reasoning), BigBench Language Identification

Table 6 reports the aggregate CORE score for each model configuration.

Figure 3 plots CORE score against active FLOPs, analogous to the BPP scaling analysis in Figure 2. At matched active FLOPs, the Remix d8 achieves a CORE score 36% above the dense power-law

prediction (0.123 vs. 0.090 predicted), and Remix d12 achieves 35% above (0.172 vs. 0.127 predicted). The Remix d8 matches the CORE score of a dense model with roughly $2\times$ the active parameters, while Remix d12 reaches a CORE level between the dense d17 and d20 baselines. These gains confirm that the BPP improvements documented in Section 4 translate to genuine downstream task performance rather than reflecting only compression-metric artifacts.

5 Discussion

Why fine-grained routing outperforms coarse MoE. Standard MoE routes entire tokens to one of K expert FFN blocks. At small-to-medium scale, each expert has limited capacity, and the discrete routing decision means most of the expert bank is unused per token. RemixedLinear's template mixing operates within each linear projection (six per block), providing $6 \times K$ effective routing decisions per block. We note that the advantage is primarily from this projection-level granularity and chunk amortization rather than from continuous vs. discrete routing per se: ablation 29E shows that top-1 chunk routing (discrete) matches soft chunk routing (29C) at 1.082 BPP. Both approaches, however, substantially outperform coarse FFN-level MoE at the scales we tested.

The optimization friction problem. Our 29-phase ablation history (Appendix A) documents a systematic pattern in early designs (Phases 4–9): multiplicative sigmoid gating that modulated individual basis features through learned, per-token routing consistently degraded training relative to the dense baseline. The final template-mixing design resolves this: routing operates over K template-level softmax weights rather than per-feature sigmoid gates, producing a much looser gradient coupling. Per-token learned routing (29A) works well in this formulation (1.087 BPP vs. 1.170 dense), though chunk-amortized routing (29C) achieves equivalent quality (1.082) at lower routing cost. The centered output gate ($1 + \tanh$, zero-initialized) further reduces friction by ensuring the model starts with dense-equivalent outputs (the intermediate LayerNorm introduces a minor deviation from exact gradient identity; see Section 5).

LayerNorm confound. RemixedLinear applies an intermediate LayerNorm between the basis projection and the mixing matrix (Eq. 1), which is absent from the dense baseline. At $K{=}1$ with no context or gates, the architecture reduces to $\mathrm{LN}(\mathbf{W}_b\mathbf{x}) \cdot \mathbf{T}_1$, which differs from dense only by this LayerNorm. The 0.002 BPP improvement of the $K{=}1$ baseline (1.168 vs. 1.170) may partly reflect this normalization benefit rather than the factored structure alone. A controlled ablation adding an intermediate LayerNorm to the dense baseline would isolate this effect; we leave this to future work.

Limitations. Our RemixedLinear scaling curve spans only three points (d4, d8, d12). While the trend is consistent, confirming these gains at d20+ scale ($\sim$1B active parameters) remains essential future work. The standard MoE comparison is limited to d4 due to compute constraints; extending this comparison to d8 and d12 would strengthen the conclusions. All experiments were conducted by an independent researcher without institutional compute resources, limiting the breadth of the scaling analysis. Training was performed across heterogeneous GPU hardware (A100, H100, H200, T4) using checkpoint resumption; dense baseline scores are from the published nanochat leaderboard [Karpathy, 2025] trained on 8$\times$H100. This heterogeneity precludes meaningful wall-clock training comparisons, though all models were trained to the same Chinchilla-optimal token budget. The total parameter count of RemixedLinear models is $K\times$ larger than dense equivalents; while active parameters and FLOPs are matched, memory footprint and inference latency are higher (Section 4.5).

6 Conclusion

We introduced RemixedLinear, a drop-in replacement for dense linear layers that dynamically composes K template matrices per chunk of tokens using content-aware routing. Through 29 phases of ablation, we identified chunk-amortized routing, quantile balancing, and centered output gating as the critical design choices that enable dynamic linear layers to outperform static baselines. At d12 scale, RemixedLinear achieves 10% lower BPP than the dense baseline at matched active FLOPs, while standard MoE underperforms the baseline entirely. Inference throughput is currently 3.7$\times$ lower than dense; the root causes have not been fully isolated but likely involve reduced arithmetic intensity from factored computation, increased HBM traffic from the larger template bank, and limited GPU utilization from chunk-wise operations. Closing this gap through kernel fusion, weight

compression, or inference-time distillation is the primary direction for future work. These results suggest that fine-grained template routing within linear projections provides a promising alternative to coarse expert routing for compute-efficient Transformers.

References

Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, volume 35, 2022.

Yann N. Dauphin, Angela Fan, Michael Auli, and David Grangier. Language modeling with gated convolutional networks. *arXiv preprint arXiv:1612.08083*, 2017.

William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.

David Ha, Andrew Dai, and Quoc V Le. Hypernetworks. In *International Conference on Learning Representations*, 2017.

Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, et al. Training compute-optimal large language models. *arXiv preprint arXiv:2203.15556*, 2022.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Keller Jordan, Yuqing Chen, Micah Goldblum, Nikunj Saunshi, and Preetum Nakkiran. Muon: An optimizer for hidden layers in neural networks. *arXiv preprint arXiv:2502.16982*, 2025.

Andrej Karpathy. nanochat: The best chatgpt that $100 can buy, 2025. URL `https://github.com/karpathy/nanochat`.

Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, Dehao Chen, Orhan Firat, Yanping Huang, Maxim Krikun, Noam Shazeer, and Zhifeng Chen. Gshard: Scaling giant models with conditional computation and automatic sharding. In *International Conference on Learning Representations*, 2021.

Jeffrey Li, Alex Fang, Georgios Smyrnis, Arjun Mahdavi, Christoph Schuhmann, LAION Laion, Samir Yitzhak Gadre, Gabriel Ilharco, Jenia Jitsev, Filip Radenovic, et al. Datacomp-lm: In search of the next generation of training sets for language models. *Advances in Neural Information Processing Systems*, 37, 2024.

NVIDIA. Climbmix: A curated pretraining dataset. 2024. Used via nanochat framework.

Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pages 9342–9351. PMLR, 2021.

Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*, 2017.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Yanqi Zhou, Tao Lei, Hanxiao Liu, Nan Du, Yanping Huang, Vincent Zhao, Andrew Dai, Zhifeng Chen, Quoc Le, and James Laudon. Mixture-of-experts with expert choice routing. *Advances in Neural Information Processing Systems*, 35, 2022.

Barret Zoph, Irwan Bello, Sameer Kumar, Nan Du, Yanping Huang, Jeff Dean, Noam Shazeer, and William Fedus. St-moe: Designing stable and transferable sparse expert models. *arXiv preprint arXiv:2202.08906*, 2022.

A Design Evolution: From CCL to RemixedLinear

The final RemixedLinear architecture emerged from 29 phases of systematic ablation, each testing a specific hypothesis about how to make context-conditioned linear layers work. This appendix summarizes the key phases and the lessons they produced.

Phase 1–2: Global Context Manager. The original design used a cross-attention mechanism to compute a global context vector for modulating all linear layers. This architecture showed strong early results but was later discovered to contain a **causal leakage bug**: the cross-attention looked into future tokens. Once the leak was fixed, the architecture failed to beat the dense baseline.

Phase 3: BPTT through context. Allowing gradient flow through the recurrent context stream (removing `.detach()`) caused gradient explosion. This established that **context states must be detached** between time steps.

Phase 4–8: Activation vs. weight modulation. DiT-style activation modulation (AdaRMSNorm), multi-scale temporal bottlenecks, cross-layer stale context, and boundary-gated updates were all tested. None matched the dense baseline. The core finding: **weight-level modulation is more expressive than activation-level modulation**, but the gating mechanism must not interfere with the primary gradient path.

Phase 9: The identity trap. When context is derived from the same $\text{norm}(\mathbf{x})$ that the FFN processes, multiplicative sigmoid gating collapses to identity. Additive residual designs (RAL) also failed despite identity-safe initialization, establishing that **zero-init does not guarantee zero friction** during training.

Phase 12–17: CKR and position routing. Causal Kernel Reparameterization (CKR) using position-only routing was the first design to beat the dense baseline (+0.02 BPP). However, this improvement could not be reproduced reliably and did not scale beyond $K{=}4$ branches. The lesson: **position conditioning has a low ceiling**, as RoPE already provides adequate positional information.

Phase 20–21: Frozen content routing. Low-Rank Composition with Frozen Basis (LRCFB) used a frozen random projection for content-dependent routing, achieving -0.025 BPP improvement. This validated that **content routing works if the routing projection does not receive gradients from the main loss**.

Phase 22–29: Template mixing and chunk routing. Building on LRCFB, we introduced learned templates with chunk routing, quantile balancing, and the centered output gate. The $1 + \tanh$ gate formulation was the final key ingredient, resolving the optimization friction that had blocked all prior sigmoid-gated designs. The canonical configuration ($K{=}8$, chunk-64, quantile routing) is the subject of this paper.

Summary of failure modes.

- **FM1 (Gradient fracturing):** In early sigmoid-gated designs, learned per-feature routing coupled with feature weight gradients degraded training. The template softmax formulation avoids this by routing at the template level rather than the feature level.
- **FM2 (Identity trap):** Context derived from $\text{norm}(\mathbf{x})$ collapses gating to identity.
- **FM3 (Rank bottleneck):** Subspace restrictions (low-rank deltas, channel splitting) underperform full-rank alternatives.
- **FM4 (Capacity tax):** At small scale, any parameter overhead for routing mechanics is proportionally too expensive.
- **FM5 (Zero-init illusion):** Starting as identity does not prevent the added pathway from disrupting training once gradients flow.

The resolution to all five failure modes was the combination of (1) template-level softmax routing (replacing per-feature sigmoid gating) to resolve FM1, with optional chunk amortization for efficiency,

(2) a detached causal context stream to avoid FM2, (3) full-rank templates to avoid FM3, (4) the centered output gate to avoid FM4 and FM5.

NeurIPS Paper Checklist

1. **Claims**

   Question: Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?

   Answer: [Yes]

   Justification: The abstract and introduction state specific quantitative results (10% BPP improvement at d12, MoE underperformance) that are directly supported by Table 3 and Figure 2.

   Guidelines:

   - The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
   - The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
   - The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
   - It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. **Limitations**

   Question: Does the paper discuss the limitations of the work performed by the authors?

   Answer: [Yes]

   Justification: Section 5 (Discussion) includes a dedicated Limitations paragraph discussing the limited number of scaling points (3), restricted MoE comparison (d4 only), single-researcher compute constraints, and higher memory footprint from $K\times$ total parameters.

   Guidelines:

   - The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
   - The authors are encouraged to create a separate "Limitations" section in their paper.
   - The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
   - The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
   - The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
   - The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
   - If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
   - While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren't acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. **Theory assumptions and proofs**

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper does not include formal theoretical results or proofs. The complexity analysis in Table 1 presents standard FLOPs counting, not a theorem.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. **Experimental result reproducibility**

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Section 4 specifies the framework (nanochat), dataset (ClimbMix), tokenizer vocabulary size, sequence length, optimizer (Muon + AdamW), training horizon (10.5$\times$ token-to-parameter ratio), and all RemixedLinear hyperparameters (K=8, chunk size 64, quantile routing). Table 2 provides exact model dimensions.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general. releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
  (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
  (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
  (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).

(d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. **Open access to data and code**

   Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

   Answer: [No] Code will be released upon acceptance.

   Justification: The implementation is built on the publicly available nanochat framework. Code for RemixedLinear will be released upon acceptance to preserve anonymity during review.

   Guidelines:

   - The answer [N/A] means that paper does not include experiments requiring code.
   - Please see the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
   - The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (`https://neurips.cc/public/guides/CodeSubmissionPolicy`) for more details.
   - The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
   - The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
   - At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
   - Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. **Experimental setting/details**

   Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

   Answer: [Yes]

   Justification: Section 4 details the optimizer, dataset, sequence length, training horizon, and all architectural hyperparameters. Table 2 provides exact model configurations at each scale.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
   - The full details can be provided either with the code, in appendix, or as supplemental material.

7. **Experiment statistical significance**

   Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

   Answer: [No]

   Justification: Due to the computational cost of training runs at the d8 and d12 scales, each configuration was run once. We acknowledge this limitation in the Discussion section. The consistency of results across three scales provides indirect evidence of reliability.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. **Experiments compute resources**

   Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

   Answer: [Yes]

   Justification: Training was performed on heterogeneous GPU hardware (NVIDIA A100, H100, H200, and T4) using checkpoint resumption across sessions. Dense baseline results at d12–d30 are from the published nanochat leaderboard trained on 8×H100. Due to the heterogeneous compute setup, wall-clock training times are not directly comparable; however, all models were trained to the same Chinchilla-optimal token budget (10.5× scaling parameters). Hardware details are discussed in the Limitations paragraph of Section 5.

   Guidelines:

   - The answer [N/A] means that the paper does not include experiments.
   - The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
   - The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
   - The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. **Code of ethics**

   Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics `https://neurips.cc/public/EthicsGuidelines`?

   Answer: [Yes]

   Justification: This work presents a novel neural network architecture component evaluated on standard language modeling benchmarks. It does not involve human subjects, sensitive data, or dual-use concerns.

   Guidelines:

   - The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
   - If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.

- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. **Broader impacts**

    Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

    Answer: [N/A]

    Justification: This work is foundational research on efficient neural network architectures. It improves compute efficiency, which has a positive environmental impact by reducing energy consumption. There is no direct path to negative societal applications beyond those inherent to language model research in general.

    Guidelines:

    - The answer [N/A] means that there is no societal impact of the work performed.
    - If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
    - Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
    - The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
    - The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
    - If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. **Safeguards**

    Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

    Answer: [N/A]

    Justification: The paper presents an architecture component, not a released pre-trained model. The models trained are small-scale ($\leq$290M active parameters) and trained on publicly available data.

    Guidelines:

    - The answer [N/A] means that the paper poses no such risks.
    - Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
    - Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
    - We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. **Licenses for existing assets**

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The nanochat framework (MIT License) and ClimbMix dataset (CC-BY-4.0) are cited. The BPE tokenizer is from the nanochat release (MIT License). All baseline architectures and methods are properly referenced.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, `paperswithcode.com/datasets` has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. **New assets**

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: No new datasets or pre-trained models are released with this submission.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. **Crowdsourcing and research with human subjects**

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: This work does not involve crowdsourcing or human subjects.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. **Institutional review board (IRB) approvals or equivalent for research with human subjects**

    Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

    Answer: [N/A]

    Justification: This work does not involve human subjects research.

    Guidelines:

    - The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
    - Depending on the country in which research is conducted, IRB approval (or equivalent) may be required for any human subjects research. If you obtained IRB approval, you should clearly state this in the paper.
    - We recognize that the procedures for this may vary significantly between institutions and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the guidelines for their institution.
    - For initial submissions, do not include any information that would break anonymity (if applicable), such as the institution conducting the review.

16. **Declaration of LLM usage**

    Question: Does the paper describe the usage of LLMs if it is an important, original, or non-standard component of the core methods in this research? Note that if the LLM is used only for writing, editing, or formatting purposes and does *not* impact the core methodology, scientific rigor, or originality of the research, declaration is not required.

    Answer: [N/A]

    Justification: LLMs were used only for writing assistance, not as a component of the core methodology. This usage does not impact the scientific rigor or originality of the research.

    Guidelines:

    - The answer [N/A] means that the core method development in this research does not involve LLMs as any important, original, or non-standard components.
    - Please refer to our LLM policy in the NeurIPS handbook for what should or should not be described.